

Vision-Based Traffic Light Control Using YOLOv8 for Detecting Moving Vehicles, Pedestrians, and Animals

Noor Al Azzawi, Russell Sullivan, Sarah Levitan

Introduction

Computer vision has become a core component of modern traffic systems, autonomous vehicles, and public safety technologies. Traditional traffic lights operate on fixed timers and are unable to respond to real-time changes on the road, which can lead to inefficiencies and unsafe scenarios especially when unexpected obstacles like pedestrians or animals appear. This project aims to address these challenges by creating a software-based smart traffic light system, serving as a working example for a Computer Vision class on how decision-making can be automated using video analysis.

The project observes video streams uploaded by the user, leveraging the state-of-the-art YOLOv8 object detection model (pre-trained on the COCO dataset) to identify vehicles, people, and animals in each frame. Key Python frameworks we used include OpenCV for video and image processing, PyTorch/TensorFlow for deep learning inference, and supporting libraries such as NumPy, Pandas, and Matplotlib for data handling and visualization. Project development and documentation are managed in GitHub using Jupyter Notebooks, ensuring each step and experiment is transparent and reproducible.

Upon detecting target objects, the system applies straightforward logical rules to switch a simulated traffic light between red, yellow, and green states, reacting immediately to the presence or absence of moving cars, pedestrians, or animals. Results show that this approach can successfully adapt traffic control in simulated environments, providing timely responses to real-world scenarios. Ultimately, the project highlights the powerful intersection of deep learning algorithms and practical urban applications, demonstrating the potential of intelligent, context-aware systems to improve road safety and traffic management while providing a clear, extensible framework for further academic exploration.

2. Methodology

2.1 Tools (see initial commit & setup: [Here](#))

- **YOLOv8 (Ultralytics):** helps you spot cars, people, dogs, and cats in pictures..
- **OpenCV:** grabs the video frames, draws boxes around stuff it sees, and puts everything together.
- **Python + Google Colab:** For coding and managing files
- **NumPy:** For motion tracking

YOLOv8 was chosen for its speed, ease of use, and comprehensive object detection support.

2.2 Initial Experiments: Image Testing (see commit: [car & animal detection logic: Here](#))

We first experimented using static images.

We uploaded images of cats crossing streets, vehicles approaching intersections, and night-time road scenes.

Using YOLOv8, we verified that:

1. The model consistently detected **cars**
2. It accurately detected **persons**
3. It identified **cats and dogs**

These tests confirmed that the pretrained YOLOv8 model was suitable for a real-time simulation.

2.3 Transition to Video Processing (commit: [added video processing logic: Here](#))

Once image detection worked, we extended the pipeline to support uploaded MP4 videos. This required additional steps:

- Extracting FPS and dimensions
- Preparing a **cv2.VideoWriter**
- Reading video frames in a loop
- Annotating frames with YOLO bounding boxes

One challenge was that file-upload code differed between **Jupyter** and **Colab**, so we added environment checks to handle both cases.

2.4 Motion Tracking (motion logic commit: [Here](#))

To simulate a dynamic traffic environment, we added a basic motion-tracking method:

- For every detected car, we computed the bounding-box centroid.
- We compared centroids between 3 consecutive frames.
- If the movement exceeded a threshold, we considered the car **moving**.

This boolean value became a key input to our traffic-light logic.

2.5 Traffic Light State Machine (logic controller commit: [Here](#))

The core of the system is a state machine with three states:

Green
Yellow
Red

Transitions:

- **Green changes to Yellow**
Triggered when a moving vehicle AND a person or pet are detected in the same frame, for 3 consecutive frames.
- **Yellow changes to Red**
Triggered by a timer (simulating a real caution phase).
- **Red changes to Green**
Activated once the required red duration has passed and no vehicles are moving while pedestrians/animals are present.

During each frame, the program:

1. Evaluates detected classes
2. Checks motion values
3. Updates timers
4. Writes a colored indicator (green/yellow/red) onto the annotated frame

3. Results (testing and validation: [Here](#))

We successfully built a working prototype of a vision-based control system. The program is able to:

Read a user-uploaded video
Detect multiple object classes
Track car movement across frames
Apply traffic-light decision logic
Overlay bounding boxes and light indicators
Produce a fully annotated MP4 video

In our tests, the system performed well on:

1. Daytime footage
2. Night-time scenes illuminated by headlights
3. Urban cat-crossing videos
4. Videos containing both cars and pedestrians

4. Limitations

Although the project was successful, several limitations remain such as:

1. **No depth estimation:**
The system cannot measure distance between objects; it only detects presence.
2. **Cats/dogs detection sometimes flickers one or two frames at a time.**

3. **Basic tracking logic:**

Motion tracking uses centroid differences; we did not use more advanced trackers (SORT/DeepSORT).

4. **No real hardware integration:**

The traffic light exists only as an on-screen indicator.

5. **Computation time depends on Colab GPU availability.**

Conclusion

This project demonstrates how computer vision can be used to create an intelligent and adaptive traffic-light system. By combining YOLOv8 object detection, motion tracking, and a custom state machine, we built software that responds to real-world situations such as animals or pedestrians crossing unexpectedly.

The project allowed us to gain hands-on experience with:

1. **Pretrained CV models**
2. **Real-time video processing**
3. **Logic design through state machines**
4. **Bounding-box annotation and visualization**
5. **Integrating multiple CV components into a working system**

Future improvements may include:

Integrating multi-object tracking (DeepSORT)
Adding distance estimation
Connecting the system to a live webcam stream
Upgrading to YOLOv9 or other newer models
Adding a GUI or real-time alert system

Overall, the prototype shows promise as a foundation for smart intersection technology focused on safety.

Latest Prototype: [Here](#)